BIRZEIT UNIVERSITY

Faculty of Engineering and Technology

Computer Science Department

DATABASE SYSTEM

COMP333

Project Report

Prepared by:

Abeer Salah Hussien -1220425

Noor Asfour -1232349

Supervised by: Dr. Ahmad Sabah

Section: 3

Date:18.1.2026

Abstract

This project presents the design and development of a relational database system for Saleh Khalaf Store. The system aims to efficiently manage the core business operations, including purchasing, sales, storage, inventory control, branches, managers, and employees. Given the large volume of daily transactions and the diversity of products, so database can be a solution which ensure data accuracy, consistency and ease of access.

The project focuses on analyzing system requirements and translating them into a well-structured database design in accordance with the defined project phases. Phase One addresses the requirements document, including the project scope and description, background information about the client, identification of the expected entities and relationships, and a description of the intended technology to be used. In addition, Phase One includes the specification for a sample query that demonstrate the required functionality of the database system. In Phase Two we build upon the first phase and concerns the preliminary design of the database through the development of an Entity-Relationship Diagram (ERD) that models the identified entities, attributes, and relationships in detail. In this phase, we added the normalization of the relational schema up to the Third Normal Form (3NF). The normalization process aims to eliminate data redundancy and preserve data integrity.

Table of Contents

Abstract 2

Project Description 5

Entities and their attributes 6

- **Branch** 6
- **Manager** 6
- **Employee** 6
- **Customer** 6
- **Person** 6
- **Supplier** 7
- **Product** 7
- **Warehouse** 7
- **Purchase_Order** 8
- **Sales_invoice** 8

Relations 10

- **Relation Flow** 11

Technology intends to use 11

Quires 12

- **Branch Queries** 12
- **Employee Queries** 12
- **Manager Queries** 12
- **Customer and Sales_Invoice Queries** 12
- **Wearhouse Queries** 12
- **Purchase_order Queries** 12
- **Product Queries** 13
- **Supplier Queries** 13

Entity-Relationship Diagram (ERD) – Before Normalization 14

Entity-Relationship Diagram (ERD) – After Normalization (1NF,2NF and 3NF) 16

Normalization Forms 17

- **First Normal Form (1NF)** 17

Second Normal Form(2NF) 17
Third Normal Form (3NF) 18

Project Description

The Saleh Khalaf Store Management System is a modern platform designed to completely upgrade and automate Saleh Khalaf Co. for Spices & Natural Products. The system doesn't only organize employees and branches it also gives customers the ability to shop online directly from the website. Through this system, customers can browse products, place online orders, and instantly receive a digital invoice for their purchase. At the same time, the system updates stock levels automatically in the warehouses, so every product movement whether it's a sale, a restock, or a new delivery from a supplier is recorded in real time. The goal of this project is to create a smart and connected system that manages everything from suppliers and warehouses to customers and sales, all in one place. It makes store operations faster, reduces mistakes, and improves both employee workflow and customer experience.

Entities and their attributes

Branch

- Branch_name
- Branch_id
- Branch_location
- Branch_status
- Branch _capacity
- Branch_phoneNum

Manager

- Manager_name
- Manager _id
- Category
- Phone_num
- Address
- Salary
- Branch_id

Employee

- Employee_id
- Category
- Salary
- Branch_id
- Warehluse_id
- password

Customer

- Customer_id
- password

Person

- Person_id
- Person_name
- Phone_Num
- Address

Supplier

- Supplier_id
- Supplier_name
- Phone_Num
- Address
- Email

Product

- Product_id
- Product_name
- Supplier_id
- Product_Satus
- Imported _quantity
- Available _quantity
- Reorder_level
- Price
- Production_date
- Expired _date
- Ptoduct_Category
- Product_Discretion
- Prodoct_unit
- Warehouse_id

Supplier_product

- Supplier_id, product_id

Warehouse

- Warehouse_id
- Branch_id
- Warehouse_location
- Warehous_capacity
- Warehous_status
- Warehouse_PhoneNum

Supplier_warehouse

- Supplier_id, warehouse_id

Warehouse_product

- Warehouse_id, product_id
- quantity

Purchase_Order

- Order_id
- Suppler_id
- Wearhous_id
- Order_date
- Order_status
- Orde_cost
- Product_id
- Product_ quantity

Sales_invoice

- Invoice_id
- Customer_id
- Branch_id
- Invoice_date
- Total_amount
- Payment_method

Invoice_product

- Product_id, invoice_id
- quantity

Branch_product

- Branch_id, Product_id
- Quantity
- Reorder_level_branch

Cart

- Cart_id
- Customer_id
- Creation_time

Cart_product

- Cart_id, product_id
- Quantity

Notification

- Notification_id
- Manager_id
- Employee_id
- Notification_type
- Massage
- Related_id
- Is_read
- Created_date

Orderrequest

- Request_id
- Product_id
- Branch_id
- Warehouse_id
- Employee_id
- Branch_manager_id
- Warehouse_manager_id
- Request_quantity
- Request_data
- Branch_manager_response_date
- Warehouse_response_date
- Req_status
- Rejection_reason
- Notes

Relations

From	To	Relation	Cardinality
Manager	Branch	Manage	1:1
Employee	Branch	Work	N:1
Customer	Sales_invoice	Have	1:N
Branch	Sales_invoice	published	1:N
Warehouse	Product	contain	N:N
Wearhouse	Branch	serve	1:1
Wearhouse	Manager	Manage	1:1
Wearhouse	Employee	work	1:N
Supplier	Wearhouse	supply	N:N
Supplier	Product	Supply	N:N
Supplier	Purchase Order	published	1:N
Warehouse	Purchase Order	published	1:N
Purchase Order	product	Have	N:1
manager	Person	Manager isA Person	isA
Employee	Person	Employee isA Person	isA
Customer	Person	Customer isA Person	isA
Customer	Cart	Have	N:N
product	Sales_invoice	published	N:N
cart	product	have	N:N
Branch	orderrequest	order	1:N
Warehouse	orderrequest	order	1: N

Product	orderrequest	order	1:N
Manager	Notification	Receive	1:N
Employee	Notification	Receive	1:N
Branch	Product	Have	N:N

Relation Flow

The Saleh Khalaf Store Management System connects all parts of the store in one smooth process. Each Branch has one Manager and many Employees, each Manager and Employee is a Person. Each Branch is served by one Warehouse, which stores many Products. Each Warehouse is managed by one Manager and can have many Employees working there. Suppliers supply many products to many warehouses and also supply many products directly. Suppliers and Warehouses can publish many Purchase Orders and each order is for one product. Customers are also Persons. Each customer can have many Sales Invoices and each invoice can include many products. Each Branch also publishes many invoices. Customers can add many products to Carts, and each cart can have many products. Branches, Warehouses and Products can make many Order Requests and each request is for one product. Managers and Employees can receive many Notifications. Each Branch can have many Products and each product can be in many branches. Warehouses contain many products and each product can be in many warehouses.

Technology we used

On the front-end side, HTML will be used to structure the layout of the web pages, CSS will handle the visual design and styling and JavaScript will manage page interactions and data handling. On the back-end side, the application will be built using the Flask framework in Python, which will manage the incoming data from the front-end side, apply the logic and generate appropriate responses. For data storage, the project will use a MySQL database to save user information. To connect between the front-end and back-end will be done by JSON as the format for sending and receiving data. Flask will interpret these JSON inputs, interact with the MySQL database through SQL Alchemy, and return the processed results back to the front-end to be shown to the user.

Quires

Branch Queries

- Retrieve details of all branches located in a specific city.
- Retrieve all branch details along with the Manager's name.
- Count the number of employees in each branch.
- Retrieve total sales amount for each branch.
- List all branches in a city with total invoices issued and total sales amount.
- Retrieve the branch with the most costumers and purchases.

Employee Queries

- Retrieve all employees at a specific branch with their category, phone number, and address, sorted by name.
- Retrieve all employees in a branch who are Managers only.
- Retrieve the name of the employee who has the highest salary in each branch.
- Count the number of employees by category in each branch.
- List employees along with the branch name and manager name.

Manager Queries

- Retrieve the names of all Managers at each branch, sorted by branch number.
- List Managers along with branch name.

Customer and Sales_Invoice Queries

- Retrieve all customers who have invoices in a specific branch.
- Retrieve all invoices (paid and unpaid) for each customer in a specific branch.
- Retrieve total sales amount for each customer in a specific branch.
- Retrieve invoice details along with customer name.

Wearhouse Queries

- Retrieve the remaining amount of product in each warehouse
- Retrieve all the warehouse filling next to the capacity and the current status of the Wearhouse.

Purchase_order Queries

- Retrieve the total purchase_order day \ month \ year with the supplier info.
- Retrieve the total sales_invoice day \ month \ year.

Product Queries

- Retrieve the remaining amount and reorder of product in each warehouse.
- Retrieve the best\ worst selling product among all branches per day\month\ yea or each branch.
- Retrieve the total number of products in each category at a given branch, sorted by supplier.

Supplier Queries

- Retrieve the names and phone numbers of all suppliers.
- Retrieve suppliers who deliver to a specific branch.
- Retrieve the supplier who provides the most products.
- Retrieve the total number of products supplied by each supplier and name of the product.
- Retrieve total supplied quantity from each supplier to each warehouse.

Entity-Relationship Diagram (ERD) – Before Normalization

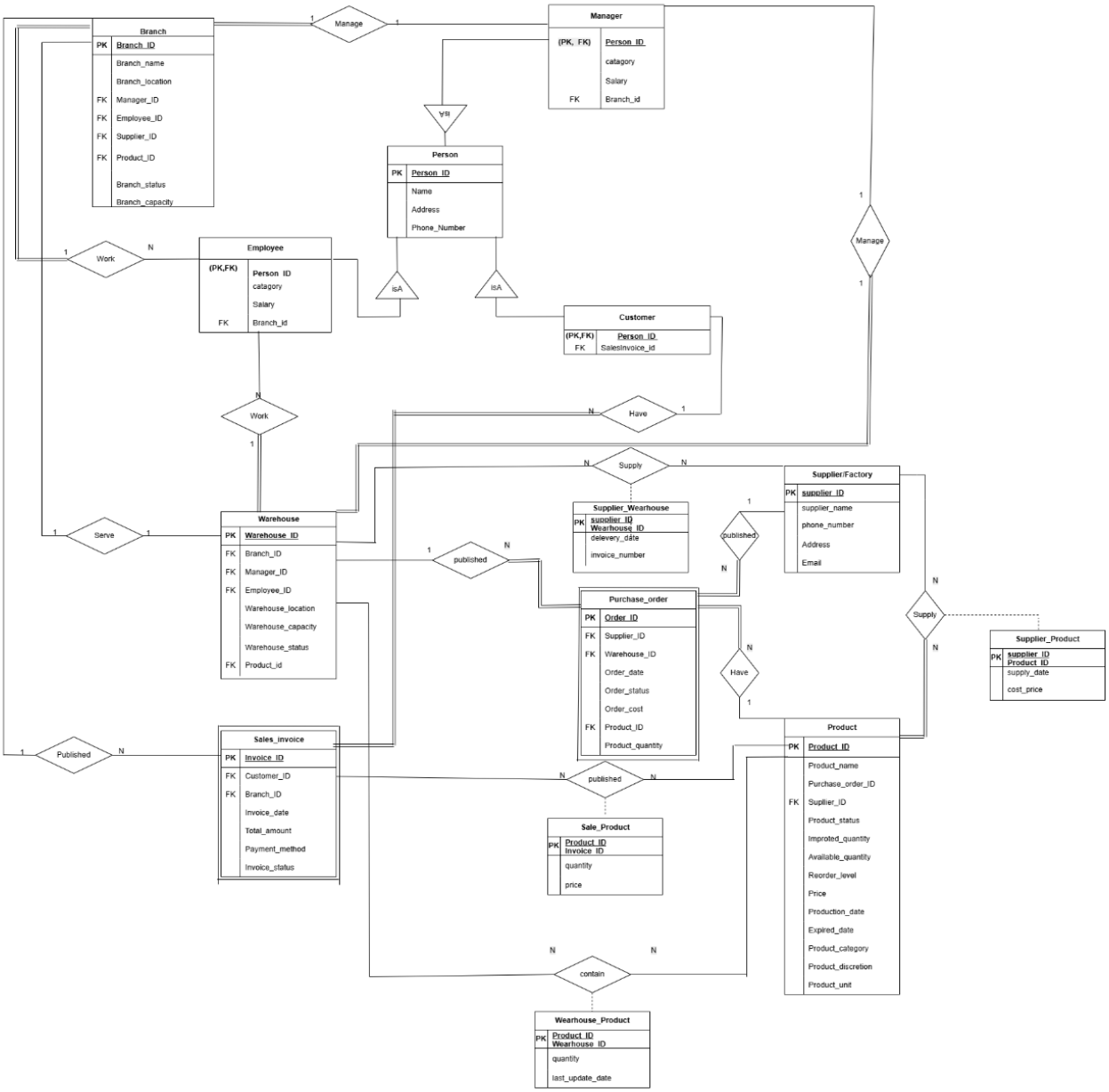

Figure 1:Before Normalization

The initial database design did not fully satisfy the normalization rules (1NF, 2NF, and 3NF). First, it didn't apply the First Normal Form (1NF) because some tables contained multiple foreign keys that represented multi-valued data. For example, the Branch table included fields such as Employee_ID, Supplier_ID, and Product_ID, which means that one branch could be

linked to many employees, suppliers, and products inside a single record, instead of using separate relationship tables. This causes data repetition and makes the table non-atomic.

Second, the design did not satisfy the Second Normal Form (2NF) because some non-key attributes were not fully dependent on the primary key, such that, in the Warehouse table attributes like Manager_ID and Employee_ID describe people rather than the warehouse itself, leading to partial dependency.

Finally, the Third Normal Form (3NF) doesn't satisfy due to transitive dependencies. In the Product table, attributes such as Supplier_ID and Purchase_order_ID were stored directly with the product, even though supplier and order information should be stored in separate tables. This means that some non key attributes depended on other non key attributes instead of the primary key which could cause data inconsistency.

Based on these issues, the database design was fixed by removing unnecessary attributes from several tables and introducing new relational tables to properly represent many-to-many relationships. This approach helped reduce the data redundancy and dependency problems.

Entity-Relationship Diagram (ERD) – After Normalization (1NF,2NF and 3NF)

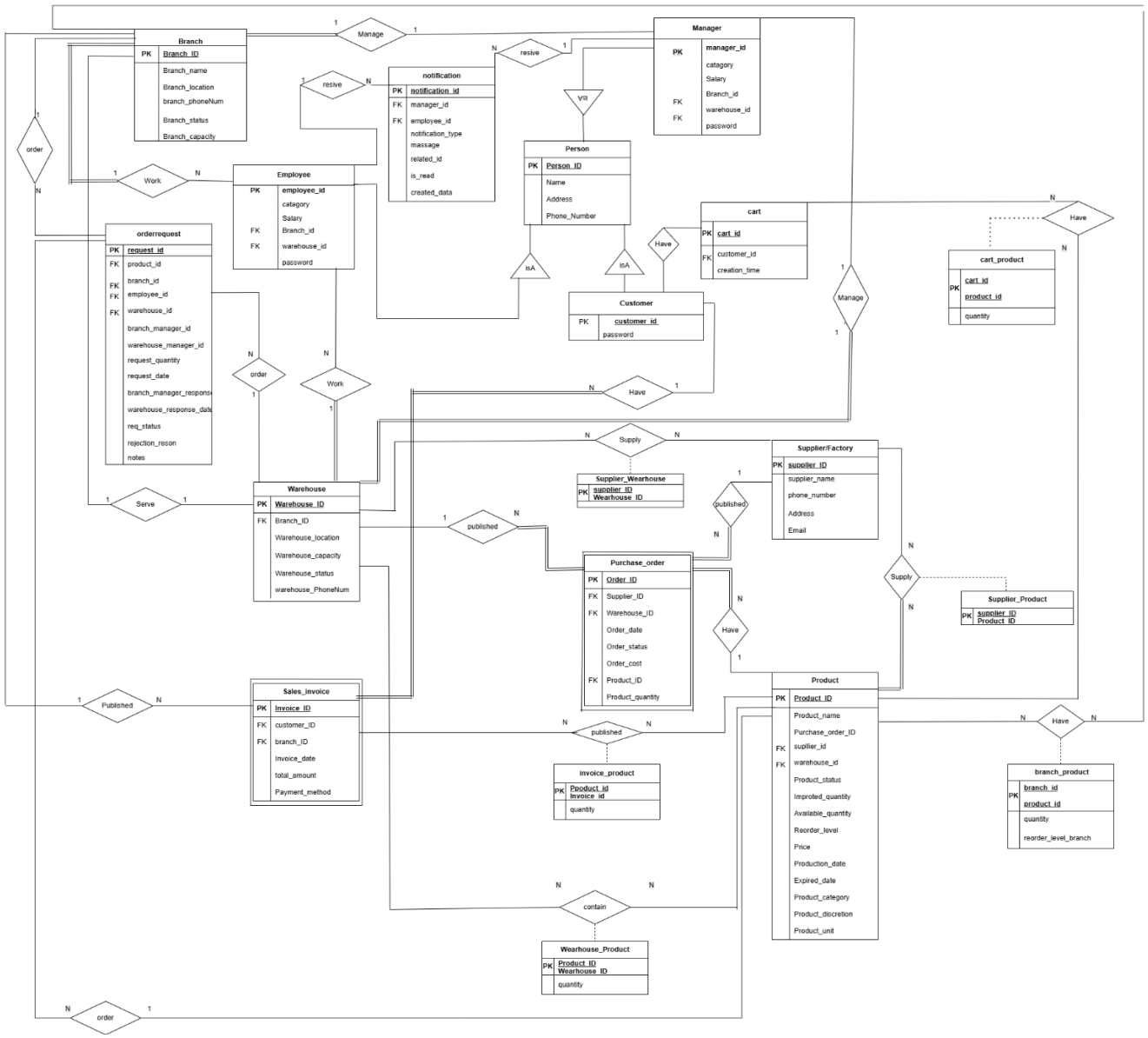

Figure 2:After Normalization

- Below, we explain the Normalization Forms which satisfied with our ERD.

Normalization Forms

First Normal Form (1NF)

First Normal Form (1NF) requires that all attributes in a relation contain atomic values, which means that each field stores a single value and not a set of values (multi value attribute). In addition, each relation must have a primary key that uniquely identifies each record in the table.

In the database design we showed in figure [1], also we represent it in the previse Phase, all tables meet the requirements of First Normal Form (1NF). Each table has a primary key, such as Branch_ID, Product_ID, Person_ID, and Invoice_ID, which uniquely identifies each record in the table. All attributes store single values only, not multiple values in one field. In addition, cases where data could be repeated, such as products linked to more than one supplier or warehouse, are handled using separate linking tables like Supplier_Product, Warehouse_Product, and Sale_Product. This approach removes repeated data and ensures that all tables follow the First Normal Form.

Second Normal Form(2NF)

The Second Normal Form (2NF) is a rule for organizing tables it's requires that all columns that are not a primary key or part of it (non-key attributes) depend on the whole primary key, not just part of it (fully dependance). Also, If the primary key has only one column, the table is automatically in 2NF, when the primary key has two or more columns (composite key), all non-key columns must depend on all key columns together. In addition, the table is already in First Normal Form (1NF).

From the tables in our database, we can note that there are tables with a single column primary key, such that

- Person ->primary key: person_id, all other columns depend on it, so it met with 2NF.
- Product ->primary key: product_id, all other columns depend on it, so it met with 2NF.

Also, we can see tables with a composite key, for example:

- invoice_product -> primary key: (invoice_id, product_id), quantity depends on both, so it satisfied with 2NF.
- warehouse_product ->primary key: (warehouse_id, product_id), quantity depends on, so it satisfied with both 2NF.

So, all tables in our database are in 2NF, no changes are needed because all non key columns depend on the full primary key.

Third Normal Form (3NF)

The ERD design follows the Third Normal Form (3NF). Hence all tables are already in Second Normal Form, and there are no transitive dependencies between non key attributes. Also, in every table, each non key attribute depends only on the primary key.

For example, in the Order_Request table, the primary key identifies each request and attributes such as, the request date, status and requested quantity all belong directly to the order request. No customer or product details are stored in this table as they are referenced using foreign keys. so, this design prevents transitive dependencies and avoids repeating the data.

In addition, for example is the Notification table, this table stores only notification-related information such as the message and date. User information is not stored directly in this table and is instead referenced using manager_id and employee_id. Because of this, all non key attributes depend only on the notification_id, which ensures that the table satisfies 3NF. This helps reduce redundancy and keeps the database consistent, which means the design fully satisfies the Third Normal Form.

P.S

For better view, we add “ERD.txt" you can follow the like then access to the ERD Design Before Normalization in page (1) and After in page (2).